大作业 · 题目

十个可选题目

题目一：并行子代理（Fan-out）

现状

- SubagentRunner::run() 是串行的，lastSteps_ 是可变成员，意味着不能被并发调用。
- Tracer::setInSubagent(bool) 是全局 bool，并发时会串号（一个子代理设 true、另一个设 false，前一个的钱被记到主循环）。
- subagentUsage 和 subagentRuns 是 shared_ptr，并发写有数据竞争。
- maybePersistLargeResult 写.af-dump/，文件名用 steady_clock::now() 做后缀，并发有概率冲突。

目标

让 Task 工具支持一次派发 N 个子代理，并发跑，各自有独立上下文、独立工具视图、独立成本，最后合并成一份结果给主模型。

设计要求

1. SubagentRunner 改成无状态：lastSteps_ 挪到 SubagentResult 里，或者每次 run() 新建一个 runner。
2. 新增 runParallel(std::vector<SubagentSpec>)：用线程池并发跑，每个子代理有自己的 AgentLoop、自己的 ToolRegistry（readOnlyView() 已经返回副本）。
3. 成本记账线程安全：subagentUsage / subagentRuns 改成 std::atomic 或加锁，或者每个子代理返回自己的 Usage 由主线程汇总。
4. Tracer::setInSubagent 改成 thread_local，否则并发串号。
5. 结果合并策略：N 份摘要怎么合并？按 goal 分组拼接？还是让模型自己选？要论证。
6. maybePersistLargeResult 的文件名加一个随机后缀或原子计数器，避免冲突。

验收标准

1. 接入实际的大模型并跑通：设 OPENAI_API_KEY，用 Task({goals: [“...”, “...”], ...}) 并发派 3 个子代理，模型能拿到 3 份摘要并给出综合回答。

2. 并发 10 个子代理跑 1 分钟，trace 里 Subagent span 的数量正好是 10，Request span 的数量是 10 的倍数，没有串号。
3. /cost 报出来的子代理 token 总和，等于各子代理 token 之和（用 trace 逐条验证）。
4. 并发跑 100 次，.af-dump/ 目录下没有文件名冲突（–selftest 里加一条 I-xx）。
5. 主循环的 spentTokens() 不包含子代理的 token（K-42 不回归）。

题目二：L3 真实检索（混合检索）

现状

- Turn::retrieved 一直是空的。ContextAssembler::assemble 里 L3 的处理逻辑写好了，但没有任何代码往 turn.retrieved 里放东西。
- L8 的“选 vs 压”只做了 repo map 部分，检索没做。
- retrieved 的预算是 forLayer(L3_Retrieved) 均分给每一条，但一条都没有。

目标

根据用户输入，从仓库里检索出最相关的代码片段，塞进 turn.retrieved，让模型在做任务前能看到“相关代码”。

设计要求

1. 嵌入模型：接一个本地嵌入模型（bge-small 或 all-MiniLM-L6-v2），把仓库的每个符号/文件块做成向量。
2. 向量库：hnswlib 或 faiss 建索引。
3. 混合检索：结合三种信号——向量相似度、repo map 的 PageRank、符号名精确匹配（grep），用 RRF 或加权融合。
4. rerank：可选，接 cross-encoder 做精排。
5. 和 Turn::retrieved 接线：在 ContextAssembler::assemble 之前检索 top-K，塞进 turn.retrieved，每条带路径、行号、分数。
6. 冷启动：第一次检索前要建索引，建索引的时间不能太长（CLI 启动不能卡超过 5 秒）。

验收标准

1. 接入实际的大模型并跑通：设 OPENAI_API_KEY，问一个需要跨文件才能回答的问题（比如“认证是在哪几个文件里处理的”），模型基于检索结果给出答案，且答案里提到的文件确实在检索

命中里。

2. –context 的 L3 层占用 > 0。
3. 写一个 A/B：开检索 vs 关检索（AF_RETRIEVE=0），同一组 10 个任务的成功率对比，开检索的成功率至少高 20%。
4. 索引构建时间 < 5 秒（1000 个文件的仓库）。
5. –selftest 里加至少 2 条 I-xx：检索结果非空、检索结果被塞进 turn.retrieved。

题目三：MCP（Model Context Protocol）支持

现状

- 工具生态只有两条路：内置工具（registerBuiltin）和 C ABI 插件（loadPlugin）。
- MCP 是 2025 年 Anthropic 提出的事实标准，当前版本完全没有。
- ToolRegistry::defs() 是静态快照，不支持运行时增减工具。

目标

让 ToolRegistry 能接入 MCP server，MCP server 暴露的工具对模型来说和内置工具没有区别。

设计要求

1. 实现 MCP client：支持 stdio 和 streamable HTTP 两种传输。
2. 协议握手：initialize → initialized → tools/list → tools/call，JSON-RPC 2.0。
3. 和 ToolRegistry 接线：MCP server 的每个工具注册成一个 Entry，execute 闭包转成 tools/call 请求。
4. 生命周期：MCP server 是长期运行的进程/连接，ToolRegistry 析构时要优雅关闭（先 shutdown 通知，再 kill）。
5. 工具描述动态变化：MCP server 可能在运行时增减工具，defs() 要能感知（用 tools/list_changed 通知，或者定期刷新）。

验收标准

1. 接入实际的大模型并跑通：设 OPENAI_API_KEY，接一个现成的 MCP server（比如 @modelcontextprotocol/ser filesystem），让模型通过 MCP 工具读文件并回答“这个文件里有几个函数”。

2. –selftest 里加一条 I-xx：MCP 工具的 defs() 和 dispatch() 和内置工具行为一致（同一个测试用例跑两遍）。
3. MCP server 异常退出时，Agent 不崩，工具调用返回错误而不是抛异常（CR-2）。
4. 写一个对比：同一个功能用 C ABI 插件实现 vs 用 MCP server 实现，改动量、灵活性、性能三项对比。
5. –cost 里 MCP 工具的调用能被 trace 记录（ToolExec span）。

题目四：完整 Tree-sitter 降级链（修复 I-20）

现状

- symbol_extract.hpp 的注释明说了：“这条线没接上，原因是接口缺字段：Message 没有 path，路径沿 Turn::retrieved → fitTo → extractSignatures 传不下来，而 tree-sitter 要按扩展名选文法。”
- ContextAssembler::extractSignatures 还是词法占位，“看起来像声明的行”。
- symbol_extract::renderSignatures 有完整实现，但只有 repo_index 在用。

目标

把 ContextAssembler::fitTo 的“签名”级从词法占位换成 tree-sitter 真解析，需要把 ContextAssembler::fitTo 的“签名”级从词法占位换成 tree-sitter 真解析，需要把 Message::path 一路传下去。

设计要求

1. 给 Message 加 path 字段（契约变更，要慎重）。可以选 std::string path（空表示不是文件内容），也可以选 std::optional<std::string> path。要在 types.hpp 顶部注释里写清楚“为什么加、加了之后哪些地方要改”。
2. 让 fitTo 能拿到 path：fitTo(content, budget, applied) 改成 fitTo(content, path, budget,applied)。所有调用点（ContextAssembler::assemble 的 L3/L4、Compactor 的 L1）都要改。
3. extractSignatures 换成调 symbol_extract::renderSignatures：这样降级链的“签名”级能用真语法分析。
4. 没有 path 的情况：工具输出没有 path，tree-sitter 不知道用哪门语言，要退回词法占位。
5. 向后兼容：L10 已经落盘的会话没有 path 字段，读回时要能容忍缺字段。

验收标准

1. 接入实际的大模型并跑通：设 OPENAI_API_KEY，用一个长文件（> 50K 字符）触发 L1 降级，模型基于降级后的签名清单能回答“这个文件里有几个类”。
2. fitTo 收到带.cpp 路径的内容，能走 tree-sitter 抽签名（–selftest 加 I-xx 验证）。
3. fitTo 收到没路径的内容，退回词法占位，不崩（–selftest 加 I-xx）。
4. L10 的会话能正常加载（path 缺失不报错，–selftest 加 I-xx）。
5. 写一个 A/B：词法降级 vs tree-sitter 降级，同一组 10 个任务的成功率对比，tree-sitter 至少高 15%。

题目五：真实 tokenizer + 预算预测

现状

- 所有预算判断都基于 estimateTokens 的粗估（4 字符 ≈ 1 token）。
- context_budget.hpp 的注释明说了：“生产实现必须接真实 tokenizer。”
- Compactor::estimateHistoryTokens 也用粗估，压缩触发时机不准。

目标

把粗估换成真实 tokenizer，重新校准所有阈值，并在请求发出前做预算预测。

设计要求

1. 接一个真实 tokenizer：tiktoken（OpenAI）、HuggingFace tokenizers（本地）、或 llama.cpp 的 tokenizer。
2. 替换所有 estimateTokens 调用点：ContextAssembler::estimateTokens、Compactor::estimateHistoryTokens。Tracer::requestChars 是字节，不用换。
3. 重新校准所有阈值：keepRecentTokens、foldThresholdChars、perMessageCapDiv、maxSummaryChars——这些默认值都是基于“4 字符 = 1 token”调的，换成真实 tokenizer 后要重新调。
4. 加“预算预测”：请求发出前，用 tokenizer 算出确切 token 数，超过 maxTokens 就提前拦截并回喂 observation。
5. 性能：tokenizer 是 BPE，每次调用有开销。要设计“只在关键点调”（比如只在组装完成后调一
6. 性能：tokenizer 是 BPE，每次调用有开销。要设计“只在关键点调”（比如只在组装完成后调一次，而不是每个 chunk 都调）。

验收标准

1. 接入实际的大模型并跑通：设 OPENAI_API_KEY，跑一个真实任务，/context 报出来的 estTok 和真实 tokenizer 的差 $< 5\%$。
2. 长会话跑 100 轮，压缩触发次数和 token 消耗的曲线合理（不会过早触发，也不会过晚触发）。用 trace 验证：压缩触发时 historyTokens 在 [0.8 * budget, 1.2 * budget] 区间内。
3. tokenizer 的开销不拖慢流式渲染（TTFT 和 L12 版本比，变化 $< 10\%$）。
4. 请求超预算时，提前拦截并回喂 observation（–selftest 加 I-xx）。
5. 写一个 A/B：粗估 vs 真实 tokenizer，同一个长会话里，压缩触发时机和压缩效果的对比。

题目六：Git Worktree 隔离的子代理

现状

- 子代理和主 Agent 共用同一个工作目录。子代理改文件，主 Agent 能看见；主 Agent 改文件，子代理也能看见。
- 并发子代理改同一个文件会冲突，当前版本没有冲突处理。
- ISandbox::exec 的 cwd 参数可以传 worktree 路径，但没有代码这么做。

目标

每个子代理有独立的 git worktree，改动在 worktree 里进行，完成后 merge 回主分支或生成 patch。

设计要求

1. 给每个子代理创建一个 git worktree：git worktree add <path> <branch>。
2. 子代理的 ToolRegistry 的 workspace 指向 worktree：Read / Write / Edit / Bash 都在 worktree 里操作。
3. 子代理完成后：把 worktree 的改动 merge 回主分支（或者生成一个 patch 交给主 Agent 决定）。
4. 清理：子代理结束后删掉 worktree（git worktree remove）。
5. 冲突处理：两个子代理改同一个文件，merge 冲突时把冲突信息回喂给主 Agent。

验收标准

1. 接入实际的大模型并跑通：设 OPENAI_API_KEY，派两个子代理分别改 a.cpp 和 b.cpp，merge 后两个改动都在。

2. 两个子代理并发改同一个文件，merge 冲突时主 Agent 能拿到冲突信息，并决定怎么处理。
3. 子代理结束后 worktree 被清理（git worktree list 没有残留，–selftest 加 I-xx）。
4. 主 Agent 的工作目录在子代理运行期间不被污染（git status 干净，–selftest 加 I-xx）。
5. 沙箱的 workspace 限制跟着 worktree 变（Bash(“echo hi > ./x”) 允许，Bash(“echo hi > ../y”) 被拦）。

题目七：完整 gitignore 语法

现状

- repo_index.cpp 的 loadIgnoreRules 只认“纯名字”条目（build/、*.o、a/b/c 一律不认）。
- 注释明说了：“完整 gitignore 语法需要通配与目录语义，本课刻意不做。”

目标

支持完整的 gitignore 语法，用 git check-ignore 作为 ground truth 验证。

设计要求

1. 支持通配：*.o、build/**、**/node_modules。
2. 支持目录语义：build/ 只匹配目录，不匹配文件。
3. 支持否定规则：!important.log。
4. 支持层级：a/b/c 只匹配这个路径。
5. 支持多级.gitignore：每个目录可以有自己的.gitignore，规则叠加。
6. 性能：如果仓库有几千个.gitignore 文件，匹配时不能每次都从头扫（用 trie 或缓存）。

验收标准

1. 接入实际的大模型并跑通：设 OPENAI_API_KEY，跑一个任务，–show-repo-map 的输出里，被.gitignore 排除的文件确实没出现（和 git check-ignore 对比）。
2. 用 gitignore 官方文档里的所有例子作为测试用例，通过率 100%。
3. 仓库有 1000 个.gitignore 文件时，RepoIndex::build 的时间 < 2 秒。
4. –selftest 里加至少 5 条 I-xx，覆盖通配、目录、否定、层级、多级。

题目八：Web UI 后端

现状

- ui/tui.hpp 只有两个后端：PlainRenderer 和 FtxuiRenderer。
- UiLoop 已经通过 EventQueue 解耦，加第三个后端不需要改核心。
- 没有浏览器可用的界面。

目标

实现一个 Web UI 后端，浏览器能看到流式输出、工具卡片、权限弹窗，且核心零改动。

设计要求

1. 实现 WebRenderer：把 UiEvent 序列化成 JSON，通过 WebSocket 推给浏览器。
2. 实现一个最小前端：HTML + JS，显示对话流、工具卡片、权限弹窗。
3. 权限弹窗的双向通信:浏览器点“允许”后,通过 WebSocket 回传给 WebRenderer::answerPermission。
4. 和 makeRenderer 接线：AF_UI=web 时返回 WebRenderer。
5. 非 TTY 时的降级：无浏览器时退回 PlainRenderer（makeRenderer 永远返回非空）。

验收标准

1. 接入实际的大模型并跑通：设 OPENAI_API_KEY，浏览器打开 http://localhost:8080，输入任务，能看到流式输出。
2. 权限弹窗能点“允许”，Agent 继续执行；点“拒绝”，Agent 拿到拒绝结果。
3. 关掉浏览器，Agent 不崩（降级到 plain，–selftest 加 I-xx）。
4. ui/tui.hpp 的 Renderer 接口零改动（CR-3）。
5. 非 TTY 时 makeRenderer(true) 仍然返回非空（I-46 不回归）。

题目九：自动记忆写入策略

现状

- appendAutoMemory 有实现，但 main.cpp 里零调用。
- MEMORY.md 只能人工维护。
- loadProjectKnowledge 能读 MEMORY.md，但没人往里写。

目标

让 Agent 在跑任务的过程中自动积累“值得记住的事实”，写进 MEMORY.md，下次会话能读到。

设计要求

1. 决定“什么时候写”：循环结束时？模型主动调用一个 Remember 工具？规则判断？要论证。
2. 决定“写什么”:模型自己总结?还是从对话里提取关键事实?要不要结构化（{fact, source,timestamp}）?
3. 去重：同一个事实写两遍怎么办？检查 MEMORY.md 里已经有了？
4. 和 loadProjectKnowledge 接线：写的记忆要被下次会话读出来。
5. 和 appendAutoMemory 的接口对齐：如果要结构化，接口要改（但不改 types.hpp）。

验收标准

1. 接入实际的大模型并跑通：设 OPENAI_API_KEY，跑一次任务，MEMORY.md 里多了至少 3 条有意义的事实，且事实和任务相关（人工检查）。
2. 第二次跑任务，loadProjectKnowledge 能读到这些事实（[记忆] 项目知识 N 字符进 L1 层的 N 增加了）。
3. 同一个任务跑两次，MEMORY.md 不重复（–selftest 加 I-xx）。
4. 写一个 A/B：有自动记忆 vs 无自动记忆，同一组 10 个任务的成功率对比，有记忆的至少高 10%。
5. 自动记忆的写入不拖慢主循环（/cost 的 wallMs 变化 $< 5\%$）。

题目十：权限门“记住选择”

现状

- DefaultPermissionGate 每次都要问用户（Ask）。
- 用户没法说“总是允许这个工具”，每次都要手动确认。
- PermissionRule 是启动时静态配置的，不能持久化用户的选择。

目标

用户在权限弹窗里可以选“总是允许 / 总是拒绝”，选择持久化，下次启动生效。

设计要求

1. 加一个 AlwaysAllow / AlwaysDeny 选项：用户在权限弹窗里可以选。
2. 持久化：把选择存到.af-permissions.json 或 session_store 里。
3. 和规则表合并：启动时读持久化的选择，加到 rules_ 里。
4. 优先级：持久化的 Allow 和显式的 Deny 冲突时，Deny 仍然赢（stable_sort 把 Deny 排最前）。
5. 生命周期：持久化的规则要不要加 expiresAt？要论证。

验收标准

1. 接入实际的大模型并跑通：设 OPENAI_API_KEY，跑一个任务，权限弹窗选“总是允许 Read”，下次跑同一个工具不再问。
2. 用户选“总是拒绝 Bash(rm)”，下次跑 rm 直接拒绝，不弹窗。
3. 显式 Deny 规则仍然覆盖持久化的 Allow（–selftest 加 I-xx）。
4. .af-permissions.json 的格式是 human-readable 的，手工编辑后能生效。
5. 权限门的 stable_sort 逻辑不回归（Deny 永远排最前，I-69 不回归）。